

Deep Learning For Genetic Clarity: *Predicting Conflict in Variant Classifications*

Ayah Elkhalel

Department of Engineering
and Computer Science
The University of Texas at
Dallas
Richardson, USA
ayahelkhalel@gmail.com

Natalie Pedigo

Department of Engineering
and Computer Science
The University of Texas at
Dallas
Richardson, USA
pedigonatalie@gmail.com

Kylie Quinney

Department of Engineering
and Computer Science
The University of Texas at
Dallas
Richardson, USA
kylie.quinney@utdallas.edu

Nawaf Reaz

Department of Engineering
and Computer Science
The University of Texas at
Dallas
Richardson, USA
nawafreaz@gmail.com

Abstract—Correctly classifying genetic variants is critical for understanding their clinical significance. However, conflicting classifications across labs can pose challenges, which can lead to misinterpretation and delayed intervention. This study aims to predict which genetic variants are more likely to receive conflicting classifications using a deep neural network. With this model, we analyze genetic variants classified as benign, likely benign, of uncertain significance, likely pathogenic, and pathogenic. This approach can enhance the ability to identify variants that are more likely to have conflicting classifications, thus flagging areas that may require further review or expert consultation by the clinic. We chose to construct a neural network due to its ability to capture complex patterns, model nonlinear relationships, and manage large datasets. Despite the complexity of the genetic data, the model achieved an accuracy of 60%. This demonstrates the potential of neural networks to flag variants likely to receive conflicting classifications, providing a valuable starting point for enhancing reliability in future genetic analyses. Improvements to the model could include incorporating additional features or optimizing the model's architecture to increase accuracy.

Keywords—Genetic variants, neural networks, classification

I. INTRODUCTION AND BACKGROUND WORK

Understanding the relationship between genetics and disease is crucial when analyzing genetic variance, a vital part of computational biology and bioinformatics. Applications of genetic sequence analysis range from developing precision medicine to identifying disease-producing mutations. However, this field poses a challenge due to the vast amount of genomic data, the minimal differences between sequences, and numerous other complexities that make it difficult to differentiate between significant variants and benign ones. Conflicting classifications often occur across categories such as benign, likely benign, and pathogenic, creating challenges for clinical interpretation. [1]

Advancements in the field have increased significantly over the past two decades, providing an abundance in the availability of sequence data. Traditional approaches, such as linear

regression, are not designed to handle such complex datasets. The solution to this challenge can be found in a sophisticated algorithm, such as deep neural networks. Deep neural networks offer powerful tools capable of modeling complex, non-linear relationships and providing valuable insights into datasets. Additionally, the integration of understandable and feature-selective deep learning models has aided in the enhancement of model accuracy.

To address the challenges presented earlier, a study done by professors at Stanford University [1] suggests an interpretable neural network that has controlled variable selection. Through the implementation of multiple synthetic features, known as knockoffs, in the input layer, the model will have more secure control over the false discovery rate. The comparison of the original features with their knockoffs will provide a trustworthy selection of features while reducing randomness. In addition, sequential layers are used to minimize computational complications, resulting in a logical and dependable analysis. This approach has proved useful in detecting causal genetic variants in Alzheimer's disease, shedding light on its use in genomics.

Another notable contribution to the field of bioinformatics is attention-based models for DNA sequence analysis. DeepDeCode, presented by professors at the University of Oxford and Earlham Institute [2], is a deep learning framework intended to predict splice sites and identify DNA sequence motifs. Using attention mechanisms, DeepDeCode can secure long-term dependencies in nucleotide sequences. This will enhance the model's ability to identify biologically significant patterns. The model implements visualization techniques, such as attention maps, offering researchers a clear understanding of the influence certain patterns have on predictions, which is vital for clinical application.

These studies provide insight into the transformative potential of deep learning genetic sequence analysis. Additionally, they shed light on the significance of balancing predictive power with interpretability, which is vital in addressing challenges such as conflicting classifications (also referred to as inconsistent categorizations). This study intends to develop a deep neural network model to analyze genetic variants and predict inconsistent categorizations. The genetic variant classification file being analyzed is provided by ClinVar, a public resource that focuses on human genetic

variants. The dataset contains 65,188 instances, and 45 features, with the goal of predicting if a variant will have conflicting classifications. Through the employment of deep neural networks, our work aims to refine the model's precision while providing accurate biological interpretation of the data. By fulfilling the objective of predicting inconsistent categorizations, this study hopes to aid in reducing the uncertainty in variant analysis.

II. THEORETICAL AND CONCEPTUAL STUDY OF TECHNIQUE/ALGORITHM

We utilized a neural network, a model inspired by the structure and function of the human brain. A neural network consists of interconnected layers and nodes (neurons), which are effective for analyzing complex data due to their ability to learn and represent patterns as well as nonlinear relationships. A neural network typically consists of an input layer, hidden layers, and an output layer [4]. The input layer receives features of the dataset (in this case, genetic variant attributes). Hidden layers perform transformations using weighted connections and nonlinear activation functions (such as ReLU and Sigmoid) to model complex relationships. The final layer then outputs the prediction of the classification of the variants. In this case, the output layer produces probabilities of each classification category (benign, pathogenic, etc.).

The training process involves three key steps: forward propagation, loss calculation, and backpropagation. At a high level, this process begins with input data that is passed through the network to calculate predictions, for which then a loss function quantifies the difference between predictions and the actual values. Gradients of the loss with respect to the network weights are then calculated and optimized via gradient descent [5]. Some key advantages of using a neural network lie in its ability to model nonlinear relationships and accommodate large datasets. Genetic variant classifications often depend on subtle, nonlinear interactions between features, which neural networks excel in capturing. This algorithm is also scalable to handle large datasets like ours that include numerous features.

Deep neural nets are also effective for other tasks such as image recognition, natural language processing, and speech recognition. Although DNNs can solve problems that were previously not feasible by traditional algorithms, there are still some challenges with this technique. There is a need for vast labeled datasets, along with high computational demands. Another challenge commonly encountered is overfitting and finding the right hyper-parameters as well as features.

III. PREPROCESSING OF DATA

The preprocessing of data can be considered equally as important as the methods used to train the model. Without clean data and a solid understanding of the data that we are working with, it becomes challenging to work with the data effectively later.

Missing Data: We assessed the extent of missing values in each feature. Some features were missing over 99% of their data, so we decided to drop those features from the table entirely. Additionally, there were features in the dataset that were specific to the hospital/database from which the information was sourced, which did not pertain to the actual variants themselves, so we dropped those as well. For the remaining missing values, we used techniques such as forward

fill, interpolate, and fill na in order to fill in missing values in the other columns.

Data Imbalance: In the early stages of creating our model (results not shown) we encountered a severe imbalance in the training data, with a large portion of negative instances (variants that had not been misclassified) and very few positive instances (variants that had been misclassified).

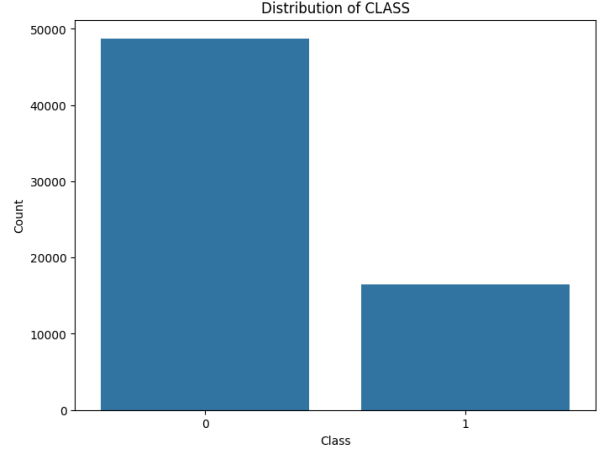


Fig. 1. Distribution of Positive and Negative Examples in the Dataset Before Resampling

This imbalance in the input data caused our model to exclusively predict a negative result. While this approach produced a relatively high accuracy (75%), it failed to address the core objective of identifying misclassified variants. To account for this issue, we used the technique of under sampling to select an equal amount of positive and negative datapoints to train the model.

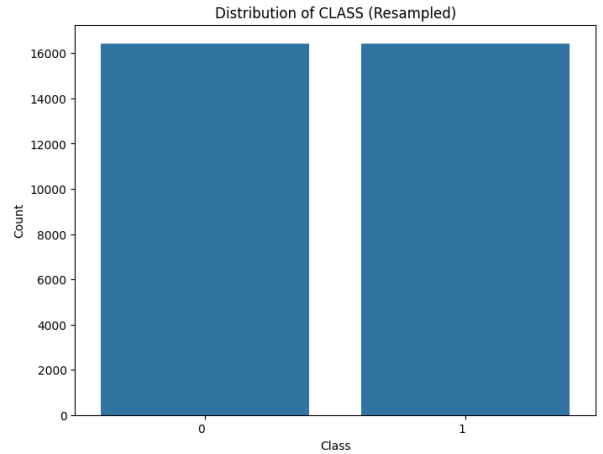


Fig. 2. Distribution of Positive and Negative Examples in the Dataset After Resampling

IV. RESULTS AND ANALYSIS

A. Experiment Log

To optimize the performance of our deep neural network for classifying genetic variants on whether they will be misclassified, we conducted a series of hyperparameter tuning experiments. The tuning process involved testing different combinations of key hyperparameters to find the optimal

configuration for our model. This approach helps ensure that the model generalizes well and performs as expected on unseen data, as well as ensuring that we are producing the best accuracy we can with our model.

The table below summarizes the results of our hyperparameter tuning experiments, including the combinations of hyperparameters tested and the corresponding evaluation metrics on the validation set. These results provide insight into the impact of different hyperparameters on the model's performance.

Iteration	Experiment Log	
	Parameters Chosen	Results
1	Neural Net: Number of FC layers: 4 (24, 250)(250, 200)(200, 50)(50, 1) Number of Activation layers: 4 (sigmoid) Neurons: 525 Error Function: RMSE Epochs: 10 Learning Rate: 0.05	Train/Test Split: 05:95 Size of dataset (neg/pos): 65188 Training Accuracy: 0.51 Test Accuracy: 0.50 Training RMSE: 0.490253 Test RMSE: 0.707469
2	Neural Net: Number of FC layers: 4 (24, 250)(250, 200)(200, 50)(50, 1) Number of Activation layers: 4 (sigmoid) Neurons: 525 Error Function: binary cross entropy Epochs: 10 Learning Rate: 0.005	Train/Test Split: 20:80 Size of dataset (neg/pos): 48754/48754 Training Accuracy: 0.51 Test Accuracy: 0.51 Training RMSE: 0.693005 Test RMSE: 0.698364
3	Neural Net: Number of FC layers: 3 (24, 12)(12, 8)(8, 1) Number of Activation layers: 3 Neurons: 45 Error Function: binary cross entropy Epochs: 15 Learning Rate: 0.01	Train/Test Split: 20:80 Size of dataset (neg/pos): 16434/16434 Training Accuracy: 0.54 Test Accuracy: 0.54 Training RMSE: 0.668838 Test RMSE: 0.680117
4	Neural Net: Number of FC layers: 3 (24, 12)(12, 8)(8, 1) Number of Activation layers: 3 Neurons: 45 Error Function: binary cross entropy Epochs: 30 Learning Rate: 0.01	Train/Test Split: 20:80 Size of dataset (neg/pos): 16434/16434 Training Accuracy: 0.59 Test Accuracy: 0.57 Training RMSE: 0.646651 Test RMSE: 0.659046

5	Neural Net: Number of FC layers: 3 (24, 12)(12, 8)(8, 1) Number of Activation layers: 3 Neurons: 45 Error Function: binary cross entropy Epochs: 40 Learning Rate: 0.005	Train/Test Split: 50:50 Size of dataset (neg/pos): 16434/16434 Training Accuracy: 0.60 Test Accuracy: 0.58 Training RMSE: 0.649471 Test RMSE: 0.651246
6	Neural Net: Number of FC layers: 3 (24, 12)(12, 8)(8, 1) Number of Activation layers: 3 Neurons: 45 Error Function: binary cross entropy Epochs: 40 Learning Rate: 0.005	Train/Test Split: 80:20 Size of dataset (neg/pos): 16434/16434 Training Accuracy: 0.61 Test Accuracy: 0.60 Training RMSE: 0.636222 Test RMSE: 0.632022

B. Analysis of Experiment Log

From the above iterations, we conclude that the sixth iteration provided the strongest results. This iteration produced a test RMSE of 0.632 and a test accuracy of 60%. This result is significant improvement as compared to the performance of the model before hyper-parameter tuning, which had a test RMSE of 0.707 and test accuracy of 50%.

Given that our makes a binary classification decision (positive or negative), an accuracy of 50% indicates that originally, our model gave results no better than guessing. However, after several rounds of hyper-parameter optimization, we achieved a notable improvement in the model's performance.

Several key factors contributed to the superior performance of the sixth iteration compared to previous attempts.

a. Train/Test Split: For the sixth iteration we used 80% of the data for training and 20% for testing, whereas in previous iterations, we reserved the majority of the data for the test phase due to concerns about overfitting. However, the model's performance on both the training and testing datasets in the sixth iteration suggests that overfitting was not a major issue, as there was no significant difference between the two in any iteration.

b. Epochs: In the earlier iterations, it was observed that the RMSE and accuracy were still improving and had not yet plateaued by the final epoch. This suggested that the model had not fully converged by the time training ended. By increasing the number of epochs, we allowed the model to train to its full potential. The extended training period allowed the model to refine its weights and adjust more effectively, contributing to the improved performance seen in this iteration.

c. Number of Neurons: In the initial iterations, we used a relatively high number of neurons in each fully connected layer, but this configuration did not yield very high results. In our later iterations we reduced the number of neurons and were able to produce better results. While it's common to suspect

overfitting when reducing the number of neurons yields better performance, there are a few reasons that we do not believe this to be the case. First, our model performed on unseen data similarly to how it did on training data, which suggests that overfitting was not the issue. Second, the switch to using more data for training than testing was made at the same time, which is likely the cause of the improvement.

Below is a plot of the Root Mean Squared Error (RMSE) for Iteration 6, tracked over each epoch during training. RMSE is used to indicate the error of the model's predictions, with lower values signifying better performance. By observing the change in RMSE over time, we can draw conclusions on whether or not the model has converged.

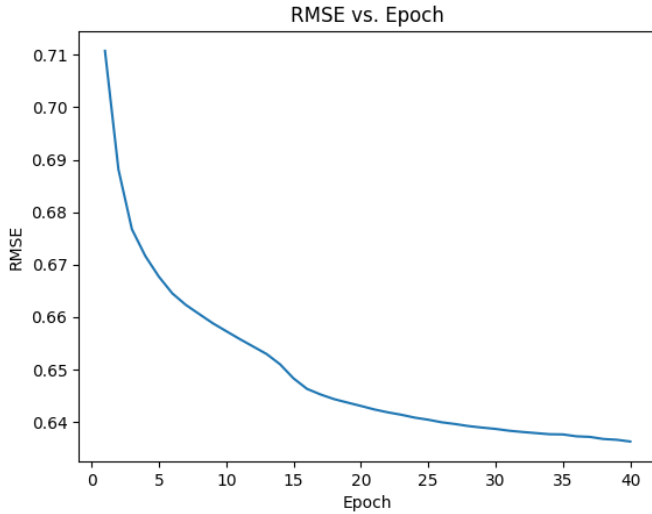


Fig. 3. Plot of RMSE vs. Epochs.

To further assess the performance of our model, we used a confusion matrix, which provides a detailed breakdown of how well the model classified the examples in the dataset. A confusion matrix is a table that compares the predicted classifications to the actual (true) classifications.

From the information presented in Figure 4, we can solve for the precision, recall, F1 score, and specificity of the model:

Precision: 0.562

Recall: 0.838

F1 Score: 0.673

Specificity: 0.373

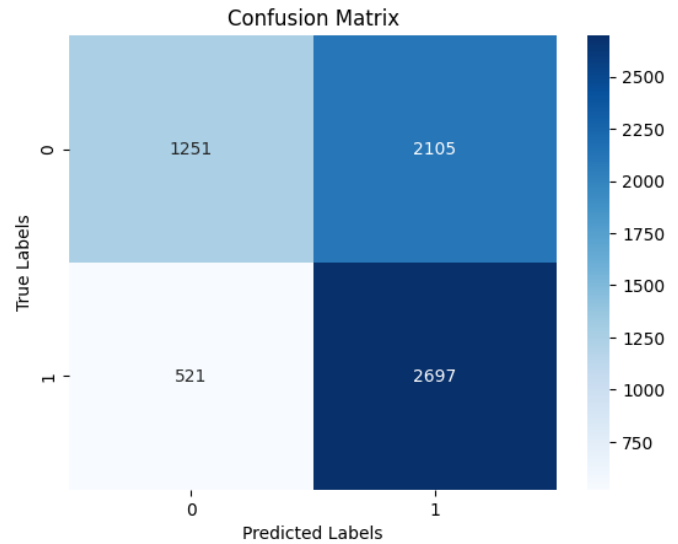


Fig. 4. Confusion Matrix.

The model has high recall, which means that it identifies positive examples, signified by a label of 1, extremely well. However, the model struggles with identifying negative examples, signified by a label of 0, as represented by the low recall. The precision suggests that when the model predicts a positive, it is somewhat reliable. These metrics suggest that the model is biased towards predicting the positive class.

V. CONCLUSION/FUTURE WORKS

Our research shows how neural networks can be used to properly identify genetic variants that would receive conflicting classifications. With an F1 score of 0.673, our model demonstrates the ability to differentiate true positives from both false positives and false negatives with relative success when it comes to the testing set. The contrast between the initial model performing at the same level as random chance before hyperparameter tuning and its improved capabilities afterward emphasize how being able to train on more of the data for more epochs with a more precise learning rate greatly improve the algorithm.

The results our algorithm has yielded could be applied to genetic variants that have not been classified. Variants that would receive different classifications from different laboratories are a cause for concern, as those classifications could lead to diverging treatment plans. This research can provide beneficial knowledge to medical professionals about the certainty of a given classification which can inform them when it comes to making a diagnosis.

Although we only applied the model in the context of genetic variants and if they were contentious, this research shows potential regarding the applicability of neural network algorithms in other research involving genetic variants. For instance, our research was specifically about binary classification, predicting if a genetic variant would be labeled differently by different laboratories. These labels themselves could also be modeled by the machine learning algorithm we developed although it would no longer be binary with five possible classifications.

Further research on deep neural network design and hyperparameter optimization could also improve the ability of our model to correctly classify genetic variants with disparate labels. For instance, our research indicates that one specific set of fewer nodes in fewer layers performed significantly better than another specific set of more nodes in more layers, but future models could test several different sets to identify the optimal number of layers and nodes per layer to achieve the highest F1 score. Integrating more complex processes such as step decay or cosine annealing that have been shown to improve performance in other machine learning algorithms could also be tested with this set of data to see if the same applies here.

The findings of our model highlight the usefulness of neural networks in genetics, as our results indicate how they can be employed with genomic data, specifically regarding its magnitude in both scale and complexity. Furthermore, it underscores the success of the algorithm through the use of hyperparameter tuning. By addressing some of the aforementioned areas, this research on neural networks be expanded and enhanced to provide additional insight in the field of genomic studies.

REFERENCES

- [1] P. H. Kassani, F. Lu, Y. L. Guen, and Z. He, "Deep neural networks with controlled variable selection for the identification of putative causal genetic variants," arXiv.org, <https://arxiv.org/abs/2109.14719> (accessed Nov. 18, 2024).
- [2] A. Poddar, V. Uzun, E. Tunbridge, W. Haerty, and A. Nevado-Holgado, "Identifying DNA sequence motifs using Deep Learning," arXiv.org, <https://arxiv.org/abs/2311.12884> (accessed Nov. 18, 2024).
- [3] P. Larranaga et al., "Machine learning in bioinformatics," OUP Academic, <https://doi.org/10.1093/bib/bbk007> (accessed Nov. 18, 2024).
- [4] "Neural Networks 101: Understanding the Basics of This Key AI Technology," *New York Tech Online College of Engineering & Computer Sciences*, Dec.13, 2023. <https://online.nyit.edu/blog/neural-networks-101-understanding-the-basics-of-key-ai-technology>
- [5] Han, Su-Hyun, et al. "Artificial Neural Network: Understanding the Basic Concepts without Mathematics." *Dementia and Neurocognitive Disorders*, vol. 17, no. 3, 1 Sept. 2018, pp. 83-89, [www.ncbi.nlm.nih.gov/pmc/articles/PMC6428006/](https://doi.org/10.12779/dnd.2018.17.3.83), <https://doi.org/10.12779/dnd.2018.17.3.83>.